

## **Programação Aplicada**

# **Interfaces Gráficos**

## **Caixas Dialogo, Menus e Elementos Multimédia**

**Marco Veloso**

[marco.veloso@estgoh.ipc.pt](mailto:marco.veloso@estgoh.ipc.pt)

# Agenda

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

**Elementos Gráficos, AWT e Applets**

**Interface Gráficos**

**Componentes de um Interface Gráfico**

**Gestão de Eventos**

**Contentores e Gestores de Posicionamento**

**Interacção com o utilizador**

**Caixas de diálogo e menus**

**Elementos Multimédia**

**Listagens e Tabelas**

**Formatações gráficas**

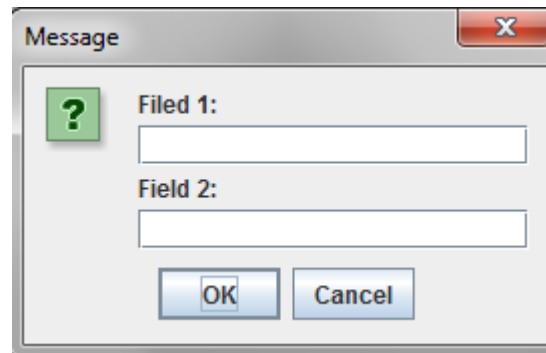
## **Interacção com o utilizador**

### **Caixas de diálogo e menus**

# Caixas de diálogo

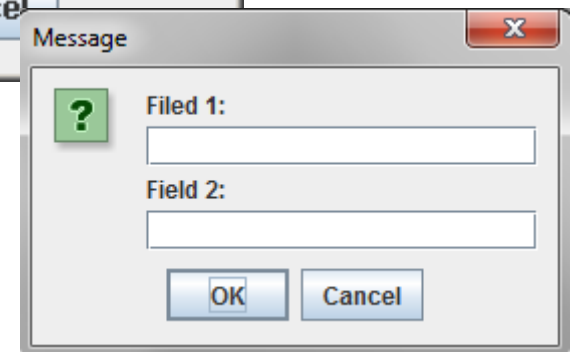
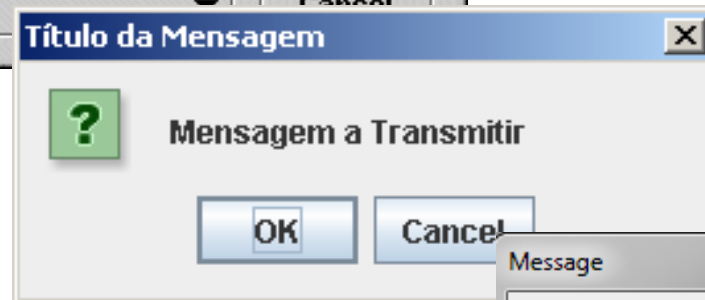
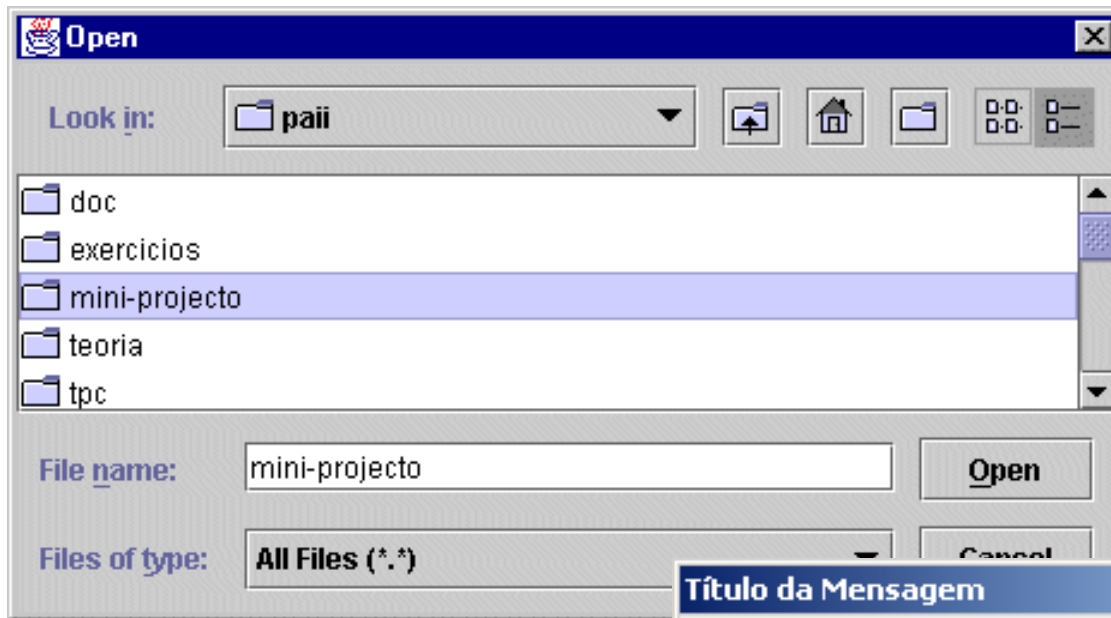
Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

## Caixas de diálogo



# Caixas de diálogo

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia



# Caixas de diálogo

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Outros componentes de uma interface gráfica que o **Swing** tem predefinidos são as **caixas de diálogo**

Uma destas **caixas de diálogo** particularmente útil para **para selecção de ficheiros (leitura ou escrita)** é definida pela classe **JFileChooser**

```
JFileChooser dLerFich = new JFileChooser();  
dLerFich.showDialog(this, "Open");           //mostra caixa de diálogo  
File file = dLerFich.getSelectedFile();      //obtem nome do ficheiro
```

Além da **construção do objecto**, usando o operador **new** como é usual, é necessário **invocar o método `showDialog()` para a caixa ficar visível**

Os parâmetros para este método são: o **objecto-pai da caixa** (deve ser uma janela (**JFrame**)), e o **título** (uma *String*).

O método **`getSelectedFile()`** é usado **para obtermos uma referência (*handle*) para o ficheiro que seleccionámos**

Podemos **especificar qual a directoria a caixa de diálogo deve abrir**, indicando essa informação como argumento do construtor

```
JFileChooser( String dir );
```

por exemplo “.” indica a **directoria corrente**,

ou através da propriedade **setCurrentDirectory**

```
setCurrentDirectory( new java.io.File(".") ).
```

Para conhecer a directoria definida recorre-se ao método **getCurrentDirectory()**

Para a **abertura de directorias** (que permite aceder a todos os ficheiros incluídos nessa pasta) é necessário definir, na classe `JFileChooser` a propriedade **`setFileSelectionMode`**.

```
setFileSelectionMode( JFileChooser.DIRECTORIES_ONLY )
```

A propriedade seleccionada no exemplo **apenas permite aceder a directorias, omitindo os ficheiros**.

É possível invocar o método **`getFileSelectionMode`** para conhecer o modo de selecção definido.

Existem três modos possíveis:

**`FILES_AND_DIRECTORIES`** (opção por defeito),  
**`FILES_ONLY`** e  
**`DIRECTORIES_ONLY`**



Qualquer que seja o modo de selecção, o método **getSelectedFile** irá **devolver um objecto File**

Caso seja activado o modo **DIRECTORIES\_ONLY**, o objecto **File** recebido é uma **referência para uma directoria**, não um ficheiro

Através do método **list()**, da classe **File**, é possível obter **uma estrutura (array de Strings) com os nomes dos ficheiros do objecto File** recebido

Neste cenário é aconselhável remover a opção “*All Files*” da janela de diálogo, através de

```
setAcceptAllFileFilterUsed( false );
```

# Caixas de diálogo

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

A propriedade **setMultiSelectionEnabled** permite definir o **modo de selecção** de ficheiros ou directorias da Caixa de diálogo

O valor **true** permite **seleccionar mais do que um elemento**, o valor **false** (valor por defeito) define **apenas um elemento seleccionado** quando a caixa de diálogo é invocada

Caso o valor **false** seja escolhido, a caixa de diálogo **devolve um objecto do tipo File**, através do método

**getSelectedFile()**

Caso o valor **true** seja seleccionado, a caixa de diálogo **devolve um array de objectos File** (**File []**), sendo necessário invocar o método

**getSelectedFiles()**

# Caixas de diálogo

Interfaces Gráficos – Caixas de Diálogo, Menus e Elementos Multimédia

```
JFileChooser dLerFich = new JFileChooser( "." );
//Alternativa: dLerFich.setCurrentDirectory(new java.io.File("."));

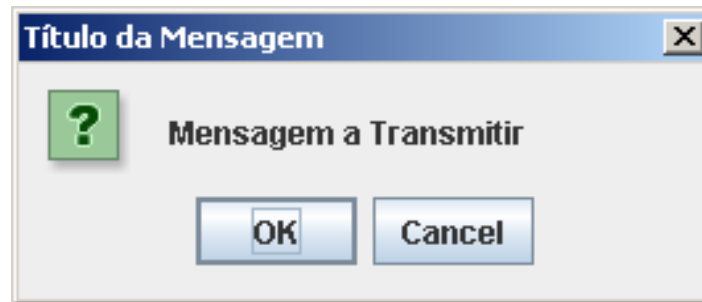
dLerFich.setDialogTitle( "Seleccionar ficheiro" );
dLerFich.setFileSelectionMode( JFileChooser.DIRECTORIES_ONLY );

// Desactiva a opção "All file types":
dLerFich.setAcceptAllFileFilterUsed( false );

if(dLerFich.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
    System.out.println("Current Dir.: " + dLerFich.getCurrentDirectory());
    System.out.println("Selected File:" + dLerFich.getSelectedFile());
} else {
    System.out.println("No Selection");
}

//Alternativa simplificada:
dLerFich.showDialog(this, "Seleccionar ficheiro");
File file = dLerFich.getSelectedFile();
String nomeDirectoria = file.getAbsolutePath();
```

## Caixas de Mensagem



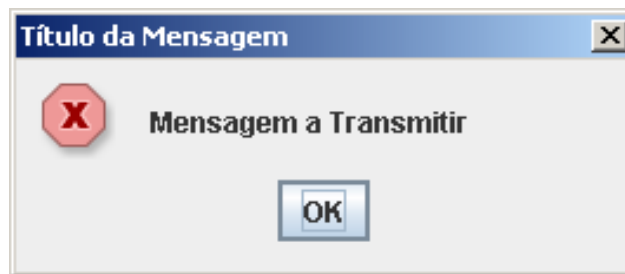
# Caixas de mensagem

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

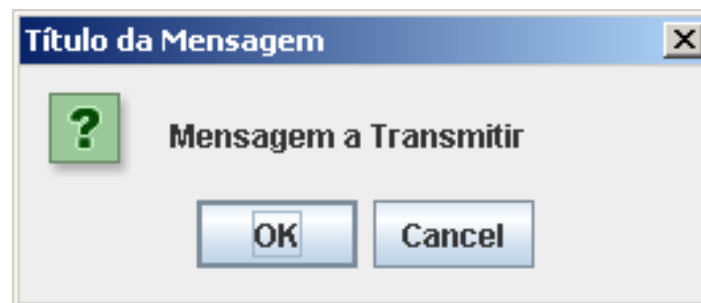
Outro tipo de caixas de diálogo permitem **disponibilizar mensagens genéricas ao utilizador e aguardar a sua acção**, definidas pela classe **JOptionPane**

Destacam-se dois tipos de caixas de de mensagem

**Caixas de Mensagem (Message Dialog)** – apresentam uma mensagem ao utilizador, mas sem acção/escolha do utilizador



**Caixas de Confirmação (Confirm Dialog)** – apresentam uma mensagem ao utilizador, aguardando uma acção/escolha do utilizador



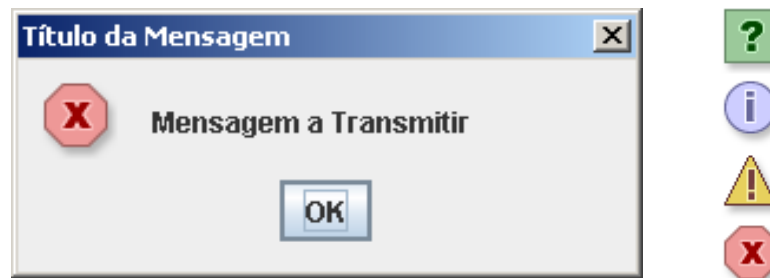
# Caixas de mensagem

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

As caixas de mensagem permitem apresentar mensagens ao utilizador (**showMessageDialog** da classe **JOptionPane**), embora não permitam a interacção com o utilizador:

```
JOptionPane.showMessageDialog( this,  
                                "Mensagem a Transmitir",  
                                "Título da Mensagem",  
                                JOptionPane.ERROR_MESSAGE );
```

E o resultado é o seguinte, sendo possível definir o tipo de mensagem a apresentar (*informação, aviso, erro, etc.*), alterando o valor do último parâmetro:



Entre outras, podemos recorrer aos tipos (definidos na classe **JOptionPane**):

OK\_OPTION (por defeito),  
ABORT\_OPTION,  
WARNING\_MESSAGE,

OK\_CANCEL\_OPTION,  
ERROR\_OPTION,  
PLAIN\_MESSAGE,

CANCEL\_OPTION,  
QUESTION\_MESSAGE,  
INFORMATION\_MESSAGE

no entanto **disponibiliza sempre o mesmo botão OK, apenas modificando o sinal de aviso**

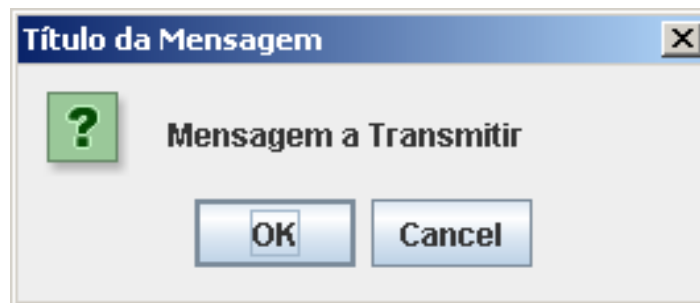
# Caixas de mensagem interactivas

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Para permitir que o **utilizador defina o rumo a seguir** (se aceita ou não uma informação, por exemplo) necessitamos de outro tipo de caixas de diálogo (**showConfirmDialog** da classe **JOptionPane**):

```
int valor = JOptionPane.showConfirmDialog(this,  
    "Mensagem a Transmitir",  
    "Título da Mensagem",  
    JOptionPane.OK_CANCEL_OPTION);
```

E o resultado é o seguinte:



Consoante o utilizador pressione em **OK** ou **CANCEL**, a variável **valor** armazenará 0 ou 2

# Caixas de mensagem interactivas

Interfaces Gráficos – Caixas de Diálogo, Menus e Elementos Multimédia

É possível **alterar a linguagem de apresentação dos botões**.

Uma abordagem será **definir a linguagem por defeito** no arranque da aplicação, definindo a propriedade `user.language`:

```
java -Duser.language=PT ClassName
```

Outra abordagem é indicar na janela gráfica o texto associado a cada botão:

```
UIManager.put("OptionPane.cancelButtonText", "Cancelar");
```

```
UIManager.put("OptionPane.noButtonText", "Não");
```

```
UIManager.put("OptionPane.okButtonText", "Confirmar");
```

```
UIManager.put("OptionPane.yesButtonText", "Sim");
```



# Caixas de mensagem interactivas

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Para além de mensagens, **as caixas de diálogo** (ou caixas de mensagem) **também podem incluir outras componentes.**

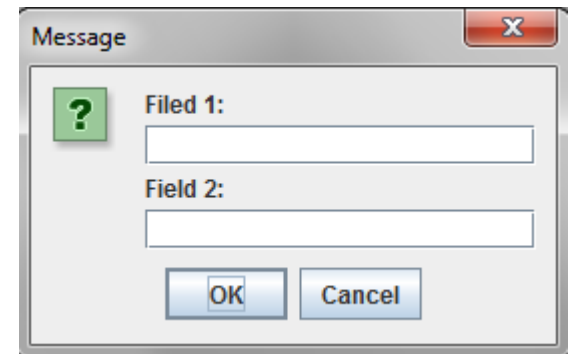
Para o efeito deve ser disponibilizado um *array* de `JComponent` com os elementos a serem visualizados:

```
JLabel      myLabel1 = new JLabel("Field 1:"),
JTextField  myText1  = new JTextField(10),
JLabel      myLabel2 = new JLabel("Field 2:"),
JTextField  myText2  = new JTextField(10)
```

```
JComponent[] inputs =
    new JComponent[] { myLabel1,
                        myText1,
                        myLabel2,
                        myText2  };
```

```
int valor = JOptionPane.showConfirmDialog(    null,
                                              inputs,
                                              "Message",
                                              JOptionPane.OK_CANCEL_OPTION);

String text = myText1.getText();
```



# Caixas de mensagem

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Existem outros tipos de caixas de diálogo, com diferentes fins e utilizações (e.g. **showInputDialog**; **showOptionDialog**)

Os próprios métodos **showConfirmDialog()** e **showMessageDialog()** apresentam diferentes assinaturas, com diferente número e tipo de parâmetros de entrada

As classes **JOptionPane** e **JDialog** são responsáveis pela manipulação dos diversos tipos de caixas de diálogo disponibilizadas pelo *Java Swing*

# Caixas de mensagem

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

As **caixas de diálogo** são igualmente úteis para **controlar a execução da aplicação**, especialmente em ciclos que exigem **abertura de novas janelas num ciclo**

Caso seja necessário iterar sobre vários elementos, e para cada elemento apresentar uma nova janela (e.g. formulário), devendo a **aplicação principal aguardar em cada iteração** que o utilizador termine a nova janela, a nova janela deve **herdar da classe `JDialog`** e não de **`JFrame`** sendo disponibilizada uma referência da janela principal que invoca a caixa de diálogo

Na classe que herda de **`JDialog`**, quando se pretender transferir a execução para a aplicação principal (e.g. o utilizador pressionou o botão “OK”), deve-se **invocar a instrução `dispose()`**

Desta forma, em cada iteração do ciclo, a execução é transferida para a nova janela (**`JDialog`**) e apenas quando a **`JDialog`** invocar a instrução **`dispose()`** é que a execução é transferida novamente para a aplicação principal

# Caixas de mensagem

## Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

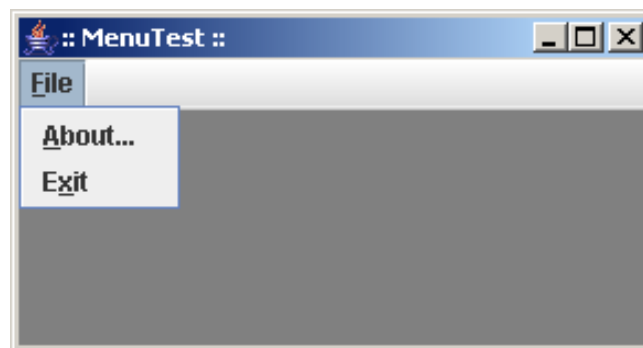
```
// Janela de diálogo, herda de JDialog e o construtor invoca o super construtor
public class MyLogin extends JDialog implements ActionListener {
    //... Code

    MyLogin(JFrame parent) { // recebe uma referência (parent) da app principal
        super(parent, "Login", true); // invoca o super construtor
        //... Code
    }

    public void actionPerformed(ActionEvent e) {
        if (e.getSource().equals(ok)) {
            //... Code
            dispose(); //notifica a classe principal, transferindo o controlo de execução
        }
    }
}

// classe principal, em cada iteração o ciclo invoca uma nova janela de diálogo e aguarda
While ( condition ) {
    //... Code
    MyLogin login = new MyLogin(this);
    login.setVisible(true);
    //... aguarda até que a instrução dispose() seja invocada na JDialog
    //... Code
}
```

## Menus



Para criar **menus** teremos que recorrer à classe

**JMenuBar**

Cada **elemento do menu** é criado através da classe **JMenu**, enquanto cada **secção de um menu** é obtida através da classe **JMenuItem**

Para adicionar o menu à barra de ferramentas da **JFrame** recorreremos ao método

**setJMenuBar()**

A **associação de eventos** é realizada da forma tradicional, ou seja, recorrendo à classe **ActionListener**, indicando qual o **JMenuItem** a ser observado pelo gestor de eventos através do método

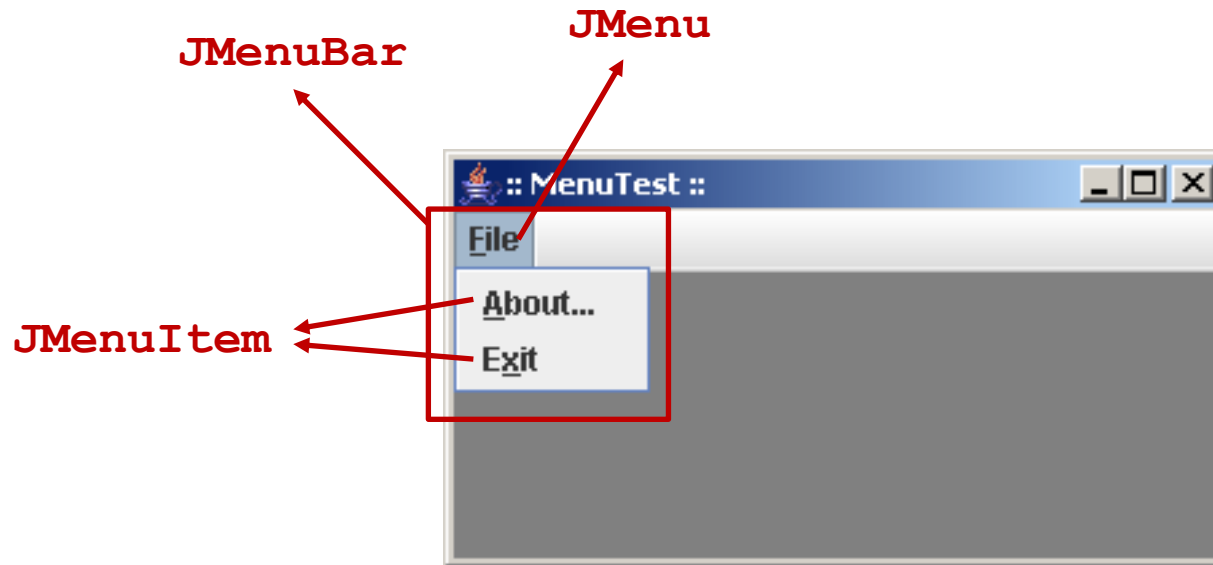
**addActionListener()**

O **tratamento do evento** (quando alguém selecciona o elemento do menu) continua a ser da responsabilidade do método

**actionPerformed( ActionEvent e )**

# Menus

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia



# Menus

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

```
JMenuBar mainBar = new JMenuBar( ); // cria menu principal

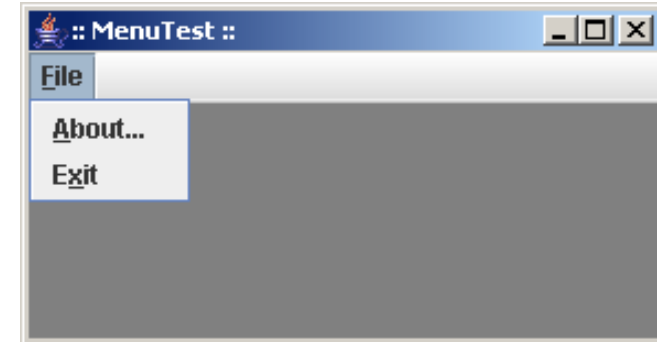
// FILE Menu
JMenu fileMenu = new JMenu ( "File" );
fileMenu.setMnemonic( KeyEvent.VK_F ); // Alt+F

// File+About
JMenuItem aboutItem = new JMenuItem( "About..." );
aboutItem.setMnemonic( KeyEvent.VK_A ); // ALT+A
aboutItem.addActionListener(this);

// File+Exit
JMenuItem exitItem = new JMenuItem( "Exit" );
exitItem.setMnemonic( KeyEvent.VK_X );
exitItem.addActionListener(this);

fileMenu.add( aboutItem );
fileMenu.add( exitItem );

mainBar.add( fileMenu );
setJMenuBar( mainBar ); // adiciona menu à aplicação
```



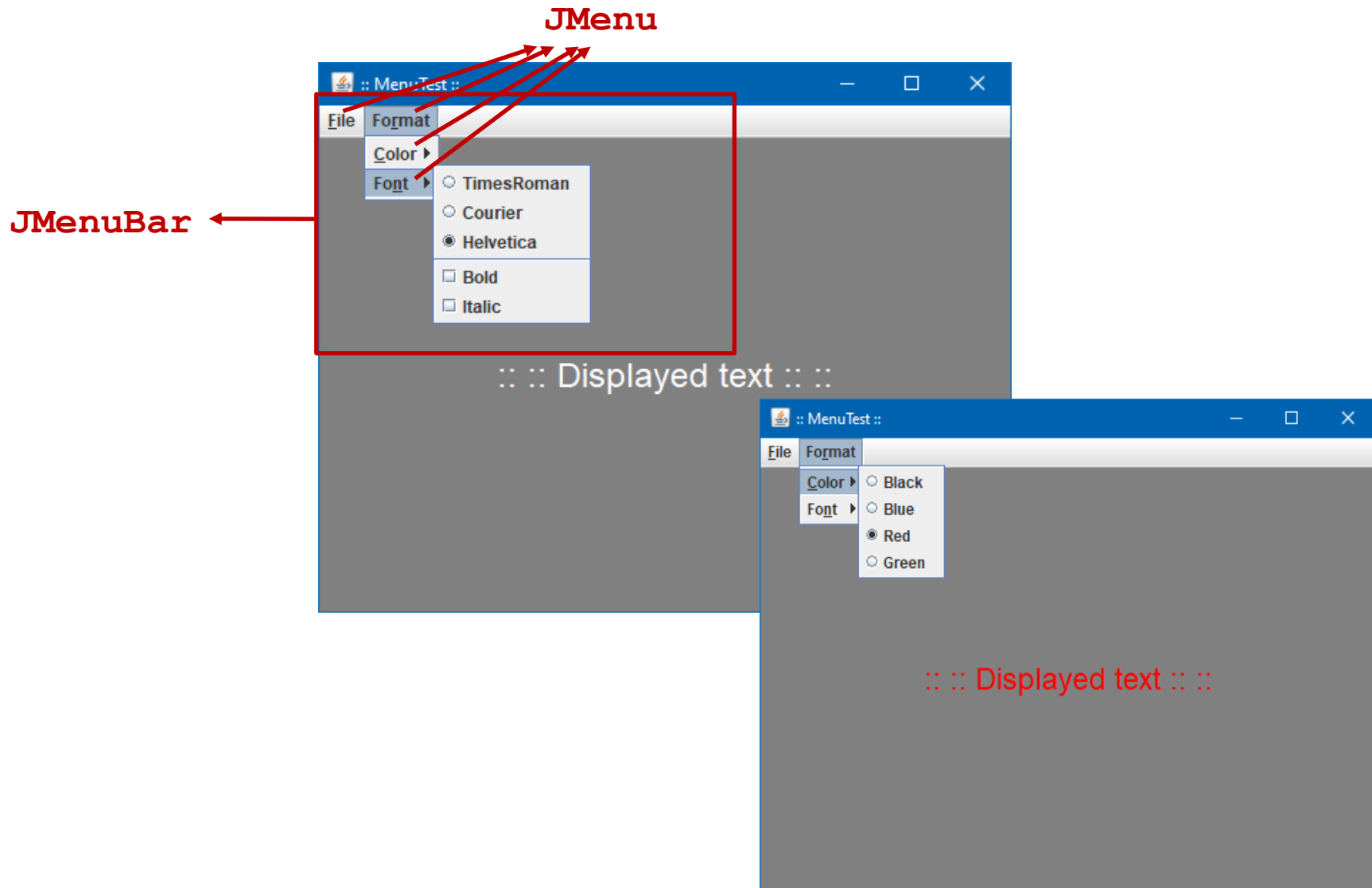


E o respectivo método `actionPerformed()` para gestão dos eventos sobre o menu:

```
public void actionPerformed((ActionEvent e) {  
  
    if ( e.getSource().equals(aboutItem) )  
        JOptionPane.showMessageDialog(  
            Menu2.this,  
            "Isto é um exemplo\n de utilização de menus",  
            "About",  
            JOptionPane.PLAIN_MESSAGE  
        );  
  
    if (e.getSource().equals(exitItem) )  
        System.exit( 0 );  
  
    repaint();  
}
```

# Menus e submenus

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

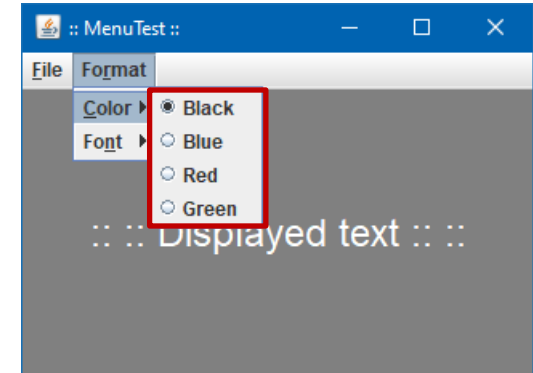


# Menus e submenus

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

```
JMenuBar mainBar = new JMenuBar( );  
(...)  
// FORMAT Menu  
JMenu formatMenu = new JMenu( "Format" );  
  
// Format+Color  
JMenu colorMenu = new JMenu( "Color" );
```

```
String [] colors = { "Black", "Blue", "Red", "Green" };  
JRadioButtonMenuItem [] colorItems = new JRadioButtonMenuItem[colors.length];  
ButtonGroup colorGroup = new ButtonGroup();  
  
for ( int i = 0; i < colorItems.length; i++ ) {  
    colorItems[i] = new JRadioButtonMenuItem( colors[i] );  
    colorMenu.add( colorItems[i] );           // adiciona os RadioButton  
    colorGroup.add( colorItems[i] );          // agrupa-os  
    colorItems[i].addActionListener(this);    // adiciona evento a cada botão  
}  
colorItems[0].setSelected( true );           // opção por defeito  
  
formatMenu.add( colorMenu );
```



# Menus e submenus

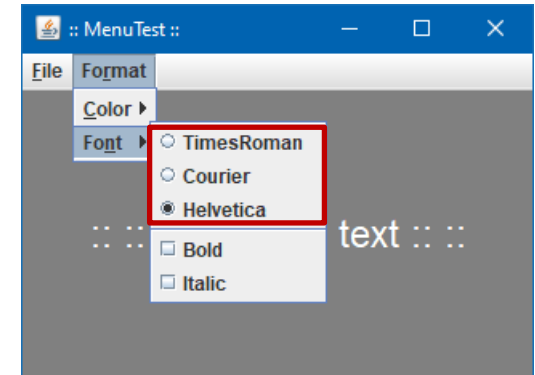
Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

```
// Font (Format+Syle)
JMenu fontMenu = new JMenu( "Font" );

String [] fontName = { "TimesRoman", "Courier", "Helvetica" };
JRadioButtonMenuItem [] fontItem = new JRadioButtonMenuItem[fontName.length];
ButtonGroup fontGroup = new ButtonGroup();

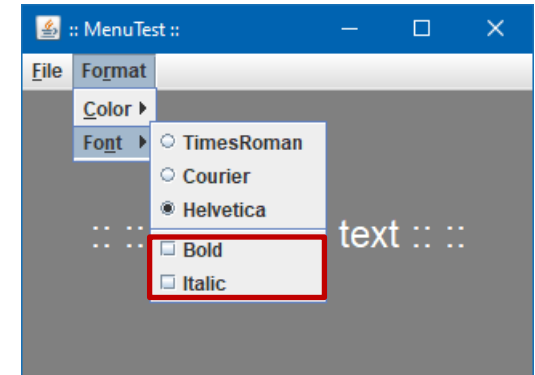
for ( int i=0; i < fontItem.length; i++ ) {
    fontItem[i] = new JRadioButtonMenuItem( fontName[i] );
    fontMenu.add( fontItem[i] );
    fontGroup.add( fontItem[i] );
    fontItem[i].addActionListener(this);
}
fontItem[2].setSelected( true );

fontMenu.addSeparator();
```



# Menus e submenus

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia



```
String [] styleNames = { "Bold", "Italic"};
JCheckBoxMenuItem [] styleItems = new JCheckBoxMenuItem[styleNames.length];

for ( int i = 0; i < styleItems.length; i++ ) {
    styleItems[i] = new JCheckBoxMenuItem( styleNames[i] );
    fontMenu.add( styleItems[ i ] );
    styleItems[i].addActionListener(this);
}

formatMenu.add( fontMenu );

// Menu principal
mainBar.add( formatMenu );
```

## **Elementos Multimédia (Imagens, Ícones, Sons) em Frames**

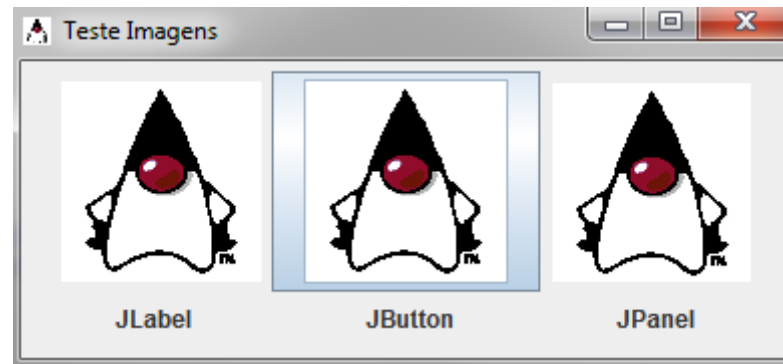
## Elementos Multimédia em *Frames*: Imagens

# Integração de imagens (*frames*)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

A integração de imagens em *frames* recorre a diferentes abordagens e componentes:

- **JLabel** (integra-se a imagem através de um **ImageIcon**);
- **JButton** (integra-se a imagem através de um **ImageIcon**);
- **JPanel** (necessário reescrever o método **paint**);





# Integração de imagens (frames – JLabel, JButton)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Para as componentes `JLabel` e `JButton`, no essencial, o processo requer a **criação de um objecto `Image` (que carrega a imagem a partir do sistema de ficheiros)**, sendo adicionado às várias componentes através da classe `ImageIcon`:

```
Image image = Toolkit.getDefaultToolkit().getImage("images/pic.jpg");

JLabel myLabel = new JLabel ( new ImageIcon(image) );
JButton myButton = new JButton( new ImageIcon(image) );
```

Alternativamente, podemos adicionar a imagem **após a criação do objecto**:

```
JLabel myLabel = new JLabel( "Lorem Ipsum" );
myLabel.setIcon( new ImageIcon(image) );
```

Para um **acesso universal** ao ficheiro de imagem:

```
Image image = Toolkit.getDefaultToolkit().getImage(
    getClass().getResource("images/pic.jpg")
);
```

# Redimensionamento de imagens

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

É possível manipular os tradicionais formatos de imagens gráficas (e.g. **JPG, GIF, PNG, BMP**)

Adicionalmente, é **possível redimensionar a imagem** a integrar na interface. Para o efeito cria-se uma nova imagem a partir do redimensionalmente da imagem original:

```
Image image = Toolkit.getDefaultToolkit().getImage("image1.jpg");  
Image scaledImage =  
    image.getScaledInstance(100, 100, Image.SCALE_DEFAULT);
```

Neste exemplo está a forçar-se a imagem a ser redimensionada para uma dimensão de 100x100

Este redimensionamento em imagens bitmap (e.g. **JPG, GIF, PNG**) pode implicar a perda de qualidade. Existem vários algoritmos para lidar com essa perda. O mais comum é através da propriedade **SCALE\_DEFAULT**

- O algoritmo **SCALE\_FAST** dá **prioridade à velocidade**;
- O algoritmo **SCALE\_SMOOTH** foca a **qualidade da imagem** em detrimento da velocidade;
- O algoritmo **SCALE\_AREA\_AVERAGING** usa uma média das áreas para garantir que a imagem final **mantém idênticas proporções** à imagem, original

# Integração de imagens (frames - JPanel)

Interfaces Gráficos – Caixas de Diálogo, Menus e Elementos Multimédia

No caso de `JPanel` a inclusão de imagens passa pela **reescrita do método `paintComponent`** ou **reescrita do método `paint`**, uma vez que estes métodos, possuindo um objecto `Graphics`, permitem o “desenho” da imagem (através da invocação ao método **`DrawImage`**)

Para o efeito temos que **criar uma nova classe que herda de `JPanel`**, invocando os super construtores e reescrevendo o método `paintComponent` ou o método `paint`

# Integração de imagens (frames - JPanel)

Interfaces Gráficos – Caixas de Diálogo, Menus e Elementos Multimédia

Inserção de imagem num painel: **opção 1, reescrever o método paint (Graphics)**

```
import java.awt.Color;      import java.awt.Cursor;
import java.awt.Dimension; import java.awt.Graphics; import java.awt.Image;
import javax.swing.JPanel;

public class MyImagePanel extends JPanel {

    public Image image;

    public MyImagePanel(int sizeX, int sizeY) {
        setPreferredSize(new Dimension(sizeX, sizeY));
        setBackground(Color.GRAY);
        setCursor(new Cursor(Cursor.HAND_CURSOR));
    }

    public void paint(Graphics g) {
        if (image != null)
            g.drawImage(image, 1, 1, this);
    }

    public Image getImage() {
        return image;
    }

    public void setImage(Image image) {
        this.image = image;
    }
}
```

# Integração de imagens (frames - JPanel)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Ao criar a classe `MyImagePanel` (que herda de `JPanel`) inclui-se o método `setImage()` para permitir definir a imagem a associar ao painel. Em alternativa, poderia ser associada a imagem através do construtor da classe.

Para integrar imagens através de um painel,

1. cria-se um objecto `JPanel` (1) usando a nossa classe (que implementa `JPanel`);
2. invoca-se o método `setImage()` (2);
3. e por fim invoca-se o método `repaint()` (3)

```
Image image = Toolkit.getDefaultToolkit().getImage(image1.jpg);
```

```
MyImagePanel myPanelDialog = new MyImagePanel (100, 100);    // 1a  
//JPanel      myPanelDialog = new MyImagePanel (100, 100);    // 1b
```

```
myPanelDialog.setLayout( new FlowLayout( ));
```

```
myPanelDialog.setImage( image );                                // 2
```

```
myPanelDialog.repaint();                                        // 3
```

# Integração de imagens (frames - JPanel)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Semelhante à opção 1, foi criada uma classe `MyImagePanel12` (que herda de `JPanel`). Como exemplo alternativo, ao invés de disponibilizar um método `setImage()`, a passagem do objecto `Image` é realizada através do construtor da classe.

Tal como no exemplo anterior, para integrar imagens através de um painel, cria-se um objecto `ImagePanel12` passando o objecto `Image` como argumento:

```
Image image = Toolkit.getDefaultToolkit().getImage(image1.jpg);  
  
MyImagePanel12 myPanelDialog = new MyImagePanel12(image);  
myPanelDialog.setLayout(new FlowLayout());
```

# Integração de imagens como background (frames - JPanel)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

```
import java.awt.Graphics; import java.awt.Image; import java.awt.LayoutManager;
import javax.swing.ImageIcon; import javax.swing.JPanel;

public class MyImagePanel extends JPanel {

    private String imageFile = "/images/backgroundFile.jpg";

    public MyImagePanel() {
        super();
    }

    public MyImagePanel(String aImage) {
        super();
        this.imageFile = aImage;
    }

    public MyImagePanel(LayoutManager aLayout) {
        super(aLayout);
    }

    public void paintComponent(Graphics g) {
        /*create image icon to get image*/
        ImageIcon icon = new ImageIcon(getClass().getResource(imageFile));
        Image image = icon.getImage();

        /*Draw image on the panel*/
        super.paintComponent(g);

        if (image != null)
            g.drawImage(image, 0, 0, getWidth(), getHeight(), this);
    }
}

// Invocação (sem definir imagem): JPanel myPanel = new MyImagePanel(new FlowLayout());
// Invocação (definindo a imagem : JPanel myPanel = new MyImagePanel("/images/backgroundFile.jpg");
```

# Integração de imagens (frames - Resumo)

Interfaces Gráficos – Caixas de Diálogo, Menus e Elementos Multimédia

Resumindo, a inserção de imagens em **frames** implica carregar a imagem para um objecto **Image**:

```
File file = new File ("images/image.jpg");  
Image image = Toolkit.getDefaultToolkit().getImage( file.toString() );
```

Para um **acesso universal** ao ficheiro de imagem (analisado de seguida):

```
Image image =  
    Toolkit.getDefaultToolkit().getImage(  
        getClass().getResource( file.toString() ));
```

E inseri-lo numa componente. No caso de um **JPanel** é necessário criar uma nova classe que herde de **JPanel** e reescreva o método **paint(Graphics)** ou **paintComponent()**, para invocar o método **drawImage()**.

```
MyImagePanel myPanelDialog = new MyImagePanel(100, 100);  
myPanelDialog.setImage(image);  
myPanelDialog.repaint();
```

Em alternativa ao uso de **JPanel**, podemos usar outras componentes (**ImagePanel**, **JButton**, **JLabel**) definido o respectivo ícone (**ImageIcon**):

```
JButton myButton = new JButton( new ImageIcon(image) );  
JLabel myLabel = new JLabel ( new ImageIcon(image) );
```



## Elementos Multimédia em Frames: Ícones

# Integração de ícones na aplicação

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Para **adicionar uma imagem como um ícone à aplicação**, no canto superior esquerdo da janela e na **barra de tarefas** devemos recorrer ao método **setIconImage()** no **main()** da aplicação:

```
public static void main(String[] args) {  
    JFrame janela = new NomeClasse();  
    janela.setIconImage(  
        Toolkit.getDefaultToolkit().getImage("Image.jpg"));  
    (...)  
}
```

Ou de forma universal:

```
janela.setIconImage(  
    Toolkit.getDefaultToolkit().getImage(getClass().getResource("image.jpg")));
```



# Integração de imagens através de ícones

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Como vimos, para integrarmos imagens numa aplicação é comum recorrer ao objecto **ImageIcon** sobre um elemento gráfico (e.g. **JLabel**)

```
String iconPath    = "images/logo.png";
JPanel panelImage = new JPanel( new BorderLayout() );
JLabel labelIcon   = new JLabel( new ImageIcon(iconPath) );
//labelIcon.setIcon( new ImageIcon(iconPath) );
labelIcon.setHorizontalAlignment( JLabel.LEFT );
panelImage.add( labelIcon );
```

Alternativa:

```
ImageIcon icon = createImageIcon("images/myFile.gif", "Lorem Ipsum");
myLabel1 = new JLabel("Image and Text", icon, JLabel.CENTER);
...
myLabel3 = new JLabel(icon);
```

O compilador vai procurar os ficheiros (imagens) à directoria `/images` que deverá ser criada na raiz do projecto (em eclipse)

Neste caso não é possível integrar os ficheiros (imagens) em ficheiros JAR

# Integração de imagens através de ícones

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Ao incluir uma imagem num elemento (e.g. `JLabel`, `JButton`) o texto desse elemento deixa de ser visível, uma vez que é necessário definir a posição do texto relativamente ao elemento, através dos métodos

- `setHorizontalTextPosition()` e
- `setVerticalTextPosition()`

Existem vários métodos, sendo o seguinte o mais simples. Por exemplo, para uma `JLabel`:

```
Image myImage =  
    Toolkit.getDefaultToolkit().getImage("images/myFile.jpg");  
  
JLabel myLabel = new JLabel();  
myLabel.setIcon( new ImageIcon(myImage) );  
myLabel.setText( "Lorem Ipsum" );  
myLabel.setHorizontalTextPosition( JLabel.CENTER );  
myLabel.setVerticalTextPosition( JLabel.CENTER );  
myLabel.setBorder( BorderFactory.createLineBorder(Color.black) );  
  
container.add( myLabel );
```

# Integração de imagens através de ícones

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Para o caso de um `JButton`, procede-se de igual forma para incluir uma imagem (como ícone) e texto sobre essa imagem, **definindo a posição do texto relativamente à imagem**:

```
Image myImage =  
    Toolkit.getDefaultToolkit().getImage("images/myFile.jpg");  
  
JButton myButton = new JButton( new ImageIcon(myImage) );  
myButton.setText( "Lorem Ipsum" );  
myButton.setHorizontalTextPosition( JLabel.CENTER );  
myButton.setVerticalTextPosition( JLabel.CENTER );  
myButton.addActionListener( this );  
  
container.add( myButton );
```

# Integração de imagens através de ícones

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Em alternativa ao uso de imagens para ícones, podemos “**desenhar**” os nossos ícones, definindo a classe **ColorIcon**:

```
public class ColorIcon implements Icon {
    private Color color;
    private int width;
    private int height;

    public ColorIcon(Color color, int width, int height){
        this.color = color;
        this.width = width;
        this.height = height;
    }
    public int getIconWidth(){
        return width;
    }
    public int getIconHeight(){
        return height;
    }
    public void paintIcon(Component c, Graphics g, int x, int y){
        g.setColor(color);
        g.fillRect(x, y, width, height);
    }
}
```

Invocação: JLabel myLabel= new JLabel( **new ColorIcon(Color.YELLOW, 200, 150)** );

# Acesso universal a ficheiros (de imagens)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Como indicado anteriormente, **para incluir ficheiros (recursos) de um projecto em ficheiro JAR** (por exemplo imagens que a aplicação necessita) é necessário **recorrer à propriedade** `class.getResource()`

Esta pode ser formada de duas formas:

```
getClass().getResource("/images/picture1.png")
```

```
NomeDaClasse.class.getResource("/images/picture1.png") ;
```

Neste caso o compilador vai procurar os ficheiros (imagens) à **directoria** `/images` que deverá ser criada na directoria `/bin` ou `/src` do projecto (em eclipse) e não na raiz do projecto

É assim possível partilhar/incluir projectos e recursos (ficheiros, como imagens) através de JARs

Atenção que o nome da directoria começa com o símbolo `/`, ou seja `"/images/picture1.png"` e não `images/picture1.png` como anteriormente

# Acesso universal a ficheiros (de imagens)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

A propriedade `class.getResource()` devolve um objecto `java.net.URL`, que pode ser usado para criar um objecto `Image` que representa a imagem importada

Esse objecto `Image` pode depois ser atribuído como ícone de um elemento gráfico (e.g. `JPanel`)

```
String iconPath = "/images/nu_fulllogo_resized.png";
JPanel panelImage = new JPanel(new BorderLayout());
java.net.URL url = NomeDaClasse.class.getResource(iconPath);
Image image = Toolkit.getDefaultToolkit().getImage(url);
JLabel labelIcon = new JLabel(new ImageIcon(image));
//labelIcon.setIcon(new ImageIcon(iconPath));
labelIcon.setHorizontalAlignment(JLabel.LEFT);
panelImage.add(labelIcon);
```



# Acesso universal a ficheiros (de imagens)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Em alternativa à instrução

```
java.net.URL url = NomeDaClasse.class.getResource(iconPath) ;  
Image image = Toolkit.getDefaultToolkit().getImage(url) ;
```

que devolve o objecto URL usado como argumento para a obtenção da imagem, podemos formar uma única instrução:

```
Image image =  
    Toolkit.getDefaultToolkit().getImage(getClass().getResource(iconPath)) ;
```

# Acesso universal a ficheiros (de imagens)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Actualizando o exemplo anterior obtemos a seguinte instrução:

```
JPanel panelImage = new JPanel(new BorderLayout());  
Image image =  
    Toolkit.getDefaultToolkit().getImage(  
        getClass().getResource(iconPath)  
    );  
JLabel labelIcon = new JLabel(new ImageIcon(image));  
labelIcon.setHorizontalAlignment(JLabel.LEFT);  
  
panelImage.add(labelIcon);
```

# Integração de imagens através de ícones

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Outra possibilidade para a inserção de imagens é **recorrer a ícones que podem ser associados a caixas de texto, botões, ou outros elementos.**

Para o efeito vamos recorrer à classe **ImageIcon**:

```
(...)  
ImageIcon icone = new ImageIcon("imagem.jpg");  
JButton botao = new JButton("Informação sobre o botão");  
botao.setBorder(new TitledBorder("Info"));  
botao.setIcon(icone);  
(...)
```

Alternativa:

```
(...)  
Icon icone =  
    new ImageIcon(getImage(getDocumentBase(), "imagem.jpg"));  
JButton botao =  
    new JButton("Informação sobre o botão", icone);  
(...)
```

# Integração de ícones na aplicação

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Para inserir a **aplicação como um processo** na **barra de tarefas** do **sistema** (*system tray*) podemos recorrer à classe

```
java.awt.SystemTray
```

É possível aceder à barra de tarefas através do método

```
SystemTray.getSystemTray(),
```

após verificar se a funcionalidade é suportada pelo sistema, através do método

```
SystemTray.isSupported()
```

É possível então inserir ou remover objectos **TrayIcons**

- Adicionar: **add**(java.awt.TrayIcon
- Remover: **remove**(java.awt.TrayIcon)

# Integração de ícones na aplicação

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Podemos posteriormente configurar o objecto na barra de tarefas para **introduzir uma imagem**

```
trayIcon.setImage(updatedImage) ;
```

inserir uma *tooltip* sempre que o rato entra na sua área de acção

```
trayIcon.setTooltip("I'm busy. Go away.");
```

e permitir que a imagem se adapta ao espaço disponível **automaticamente**

```
trayIcon.setImageAutoSize(true) ;
```

# Integração de ícones na aplicação

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

```
final TrayIcon trayIcon;  
  
if (SystemTray.isSupported()) {  
  
    SystemTray tray = SystemTray.getSystemTray();  
    Image image = Toolkit.getDefaultToolkit().getImage("tray.gif");  
  
    MouseListener mouseListener = new MouseListener() {  
  
        public void mouseClicked(MouseEvent e) {  
            System.out.println("Tray Icon - Mouse clicked!");  
        }  
  
        public void mouseEntered(MouseEvent e) {  
            System.out.println("Tray Icon - Mouse entered!");  
        }  
  
        public void mouseExited(MouseEvent e) {  
            System.out.println("Tray Icon - Mouse exited!");  
        }  
  
        public void mousePressed(MouseEvent e) {  
            System.out.println("Tray Icon - Mouse pressed!");  
        }  
  
        public void mouseReleased(MouseEvent e) {  
            System.out.println("Tray Icon - Mouse released!");  
        }  
  
    };  
    // (...)
```

# Integração de ícones na aplicação

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

```
// (...)
ActionListener exitListener = new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Exiting...");
        System.exit(0);
    }
};

PopupMenu popup = new PopupMenu();
MenuItem defaultItem = new MenuItem("Exit");
defaultItem.addActionListener(exitListener);
popup.add(defaultItem);

trayIcon = new TrayIcon(image, "Tray Demo", popup);

ActionListener actionListener = new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        trayIcon.displayMessage("Action Event",
            "An Action Event Has Been Performed!", TrayIcon.MessageType.INFO);
    }
};

trayIcon.setImageAutoSize(true);
trayIcon.addActionListener(actionListener);
trayIcon.addMouseListener(mouseListener);

try {
    tray.add(trayIcon);
} catch (AWTException e) {
    System.err.println("TrayIcon could not be added.");
}

} else { // System Tray is not supported }
```

## Elementos Multimédia em Frames: Sons



# Integração de sons (frame)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Desde as primeiras versões (JDK 1.0) que a biblioteca Java disponibiliza formas simplificadas para **leitura de ficheiros áudio**, através *package* `sun.audio.*`:

- **AudioStream** : permite carregar ficheiros áudio
- **AudioPlayer** : permite controlar a execução da música

Para o efeito carregamos o ficheiro áudio do sistema de ficheiro (`InputStream`), que será processado pela classe `AudioStream` e a sua execução fica a cargo da classe `AudioPlayer`:

```
InputStream in = new FileInputStream("C:\\myAudioFile.wav");  
AudioStream as = new AudioStream(in);  
AudioPlayer.player.start(as);
```

# Integração de sons (frame)

Interfaces Gráficos – Caixas de Diálogo, Menus e Elementos Multimédia

Adicionalmente, a *package* `sun.audio`, disponibiliza classes como `ContinuousAudioDataStream` e `AudioData` que permitem um maior controlo dos ficheiros de áudio, nomeadamente o **loop contínuo da música**

```
InputStream in = new FileInputStream("C:\\myAudioFile.wav");
AudioStream as = new AudioStream(in);

AudioData musicData = as.getData();
ContinuousAudioDataStream loop =
    new ContinuousAudioDataStream(musicData);

AudioPlayer musicPlayer = AudioPlayer.player;
musicPlayer.start(loop);
```

De frisar que estas classes apenas conseguem manipular **ficheiros de áudio não comprimidos** (e.g. wav, au, ogg Vorbis), não suportando compressões como MP3.

A Oracle/Sun disponibilizaram posteriormente a **Java Media Framework** (<https://www.oracle.com/java/technologies/javase/java-media-framework.html>), que já permite suporte para formatos adicionais, como MP3

Alternativamente, pode-se recorrer a bibliotecas externas, como o *JLayer*, *BasicPlayerAPI* ou *JACo MP3 Player*

# Integração de sons (frame)

Interfaces Gráficos – Caixas de Diálogo, Menus e Elementos Multimédia

Posteriormente, a *package* `javax.sound.sampled` introduziu novas classes para um maior controlo: `AudioInputStream`, `AudioSystem` e `Clip`:

```
InputStream in = new FileInputStream("C:\\myAudioFile.wav");
AudioInputStream as = AudioSystem.getAudioInputStream(in);

Clip clip = AudioSystem.getClip();

if ( clip != null ) {
    clip.open(as);
    clip.loop(Clip.LOOP_CONTINUOUSLY);
    clip.start();
}
```

A classe `clip` disponibiliza várias acções de controlo, nomeadamente `open`, `start`, `stop`, `loop` e `close`.

Para encerrar o objecto `clip`:

```
clip.stop();
clip.close();
clip = null;
```

# Integração de sons (frame)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

A plataforma **JavaFX** foi desenvolvida para a criação de aplicações ricas em conteúdos multimédia para a Internet. Contem as classes **Media** e **MediaPlayer** que fornecem mais funcionalidades para controlar ficheiros áudio, especialmente ficheiro codificados, como o MP3:

```
Media hit = new Media( new File("som.mp3" ).toURI().toString() );  
MediaPlayer mediaPlayer = new MediaPlayer(hit);  
mediaPlayer.play();
```

Sendo necessário realizar as importações correctas:

```
import javafx.scene.media.Media;  
import javafx.scene.media.MediaPlayer;
```

## **Elementos Multimédia (Imagens, Ícones, Sons) em Applets**

# Integração de imagens (applets)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

É possível numa `JApplet` **integrarmos imagens** em formato `*.gif` ou `*.jpg`.

Em ***Applets***, basicamente temos que **alocar espaço na memória** para a imagem através do método `getImage()` nas suas diversas variantes:

```
Image fig = getImage (
    new URL("http://localhost/Ficheiros/imagem.gif"));
Image fig = getImage (getDocumentBase(), "imagem.gif");
Image fig = getImage (getCodeBase(), "imagem.gif");
Image fig = getImage ("c://ficheiros//imagem.gif");
```

enquanto numa ***frame*** seria

```
fig = Toolkit.getDefaultToolkit().getImage(filenameOrURL);
```

Os métodos `getDocumentBase()` e `getCodeBase()` são métodos definidos nas classes `Applet` e `JApplet` que devolvem, respectivamente, o directório onde se encontra o documento HTML que contém a applet e o directório onde se encontra o ficheiro da `Applet`

# Integração de imagens (applets)

Interfaces Gráficos – Caixas de Diálogo, Menus e Elementos Multimédia

O método **getCodeBase()** serve para **localizar o ficheiro da imagem** e deve ser usado quando este está na mesma localização do ficheiro de extensão **java**, ou seja o ficheiro que contém o código do programa

É mais comum, no entanto, usar o método **getDocumentBase()** que vai **localizar o ficheiro da imagem ao sítio onde está o ficheiro de extensão html**.

Alternativamente podemos indicar o nome do ficheiro da imagem com o caminho completo. Neste caso o método **getImage()** só tem esse parâmetro

O método que carrega e desenha a imagem é o método **drawImage()**

Há vários métodos **drawImage()**, com diferentes parâmetros (*method overloading*), permitindo o usado acima desenharmos a imagem na posição x, y indicada e com o tamanho por omissão da imagem

Outros métodos **drawImage()** permitem, além disso, escalonar a imagem ao desenhá-la

# Integração de imagens (applets)

Interfaces Gráficos – Caixas de Diálogo, Menus e Elementos Multimédia

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Imag extends JApplet {

    private Image fig;

    public void init() {
        fig = getImage (getCodeBase(), "imagem.gif");
    }

    public void paint(Graphics g) {
        boolean b = g.drawImage (fig, 20, 20, this);
    }

}
```



# Integração de sons (applet)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

Também é possível numa **JApplet** tocarmos sons desde que os ficheiros onde estão pré-gravados possuam o formato \*.AU

O método que permite tocar um som é o método **play()** e tem como parâmetros a localização e o nome do ficheiro do som

```
play(new URL("http://localhost/Ficheiros/ding.au"));  
play(getDocumentBase(), "ding.au");  
play(getCodeBase(), "ding.au");  
play("c://ficheiros//ding.au");
```

Tal como para as imagens, há outro método **play()** só com um parâmetro que é o nome do ficheiro de som com o caminho completo

O código seguinte permite-nos criar uma **Applet** com um botão que, ao ser premido, toca um som

# Integração de sons (applet)

Interfaces Gráficas – Caixas de Diálogo, Menus e Elementos Multimédia

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Som extends JApplet implements ActionListener {

    private Container cont;
    private JButton b;

    public void init () {

        cont = getContentPane();    // Obtém o content pane da applet
        cont.setLayout (new FlowLayout());
        b = new JButton("Toca");    // Cria botão
        cont.add (b);              // Junta-o à applet
        b.addActionListener(this);  // Acrescenta objecto listener
    }

    public void actionPerformed (ActionEvent e) {
        play(getCodeBase() , "ding.au") ;
    }
}
```

# Integração de sons (frame)

Interfaces Gráficos – Caixas de Diálogo, Menus e Elementos Multimédia

Numa **JFrame** é possível integrar *clips* de som através da classe **AudioClip**:

```
Audioclip som = Applet.newAudioClip(getURL("dir//file.au"));
```

Esta classe disponibiliza os métodos `play()`, `stop()` e `loop()` para controlo do *clip* de áudio

Na definição anterior é usado o método `getURL()` que tem como função obter a localização correcta de um determinado ficheiro:

```
URL getURL(String pathnameRelativo) {  
    URL codeBase, completeURL = null;  
    try {  
        codeBase = new URL("file:" +  
                           System.getProperty("user.dir") + "/" );  
        completeURL = new URL(codeBase, pathnameRelativo);  
    } catch (MalformedURLException e) {  
        System.err.println(e.getMessage()); return null;  
    }  
    return completeURL;  
}
```

# Referências

Interfaces Gráficos – Caixas de Diálogo, Menus e Elementos Multimédia

## ***“Programação Orientada a Objectos”***

António José Mendes

Departamento de Engenharia Informática, Universidade de Coimbra

## ***“Java Swing”, 2ª Edição***

James Elliott, Robert Eckstein (Editor), Marc Loy, David Wood, Brian Cole  
O'Reilly, ISBN: 0596004087

## ***“Thinking in Java, ”, 4ª Edição, Capítulo 14 “Creating Windows & Applets”***

Bruce Eckel

Prentice Hall, ISBN: 0131872486

## ***“The Java Tutorial – Creating a GUI with JFC/Swing”***

Java Sun Microsystems

<http://java.sun.com/docs/books/tutorial/uiswing/index.html>

# Referências

Interfaces Gráficos – Caixas de Diálogo, Menus e Elementos Multimédia

***"The Definitive Guide to Java Swing"***, 3ª Edição

John Zukowski

Apress, ISBN: 1590594479

***"Fundamentos de Programação em Java 2"***, Capítulo 12 *"Interfaces Gráficos"*

António José Mendes, Maria José Marcelino

FCA, ISBN: 9727224237